Identifying Fires from Aerial Footage

PHAS0056- SN 23097879

Abstract

Wildfires pose an increasing global threat, where timely detection is critical to limiting ecological and human harm. Automated monitoring of aerial footage offers a scalable solution; however, video-derived datasets introduce temporal data leakage, artificially inflating validation performance. The FLAME dataset was used, applying a perceptual de-duplication (dhash), which reduced 39,375 training frames to 2,979, and a temporal block-splitting strategy to prevent leakage between the training and validation sets. The binary micro-CNN containing only 11,763 parameters, with a threshold of t=0.34 and a consecutive-frame filter of N=3, reached 86% accuracy and 97.1% Fire recall, outperforming the FLAME paper baseline by 10 percentage points. A ternary extension to three classes (Fire, No Fire, Lake) achieved 73.8% accuracy with a perfect 100% Lake recall. The ternary experiment revealed that No Fire recall in the binary model was partially inflated by visually distinct lake frames. No Fire generalisation remains the principal limitation, driven by distribution shift between the small set of de-duplicated training scenes and the diverse test environments.

1 Introduction

Wildfires are one of nature's most significant ecological and environmental threats. From 2001 to 2010, on average, 437 million hectares were burned each year [1]. Wildfires also disrupt ecosystem dynamics and biodiversity, with more than 4,400 terrestrial and freshwater species threatened by altered fire regimes [2]. Also, they contribute to increased greenhouse gas emissions, with estimates suggesting they account for 5 to 8 billion tonnes of CO2 annually [3]. The impact of wildfires extends beyond vegetation: soil degradation and erosion alter species distributions, which, in turn, affect the food chain and ecosystem stability. The risk and scale of wildfires have significantly increased over the past decades, with more frequent and severe fire events observed across continents, making monitoring and mitigation increasingly critical. In fact, in 2019/2020, the Australian bushfire season burned over 17 million hectares [4], resulting in widespread ecological damage and significant carbon emissions. Wildfires occur across large areas and long periods, affecting millions of kilometres each year [5].

Modern monitoring systems produce vast amounts of imagery from satellite sensors such as MODIS and VIIRS. There are public datasets such as the FLAME dataset [6] that provide aerial wildfire imagery collected using drones, including video frames and thermal data. These datasets enable the development of machine learning models for automated wildfire detection and monitoring.

However, image-based wildfire detection is challenging due to the variability in fire appearance, as it encompasses not only flames but also smoke and terrain. Certain non-fire features can exhibit visual characteristics similar to fire, increasing classification difficulty. A critical challenge in wildfire detection, especially from video footage, is the high intra-frame image similarity, which introduces data leakage in the dataset, leading to artificially high validation performance while reducing generalisation to unseen test data. The objectives of this project are: (1) to develop a machine learning model for wildfire classification as

a binary classifier distinguishing between images containing fire and those without, and (2) to develop a tertiary model that can further distinguish among images with fire, no fire, and lakes. These models aim to improve early wildfire detection and accurately classify key environmental features.

2 Dataset

The dataset used for this machine learning project is based on aerial wildfire imagery from the FLAME dataset. The data were collected using drones during controlled burn experiments in forest environments. The imagery was captured using both the Zenmuse X4S and Phantom 3 cameras. The dataset consists of image frames extracted from video recordings. In total, 39,375 frames captured using the Zenmuse camera were used for training, while 8,617 frames captured using the Phantom camera were reserved for testing (as shown in Table 1). Each frame is labelled as either 'fire' or 'no fire'.

Additionally, within the train data, a subset of frames was labelled as 'lake' around 11847 frames. The lake frames were split into a training and test set to construct a dataset for a multi-class classification task, enabling the model to distinguish between fire, no-fire, and lake environments. The presence of lake images introduces a potential source of bias in the binary classification task, as they are disproportionately represented in the no-fire class.

The images are provided in JPEG format with 254×254 pixels.

Camera	**Format**	**Frames**	**Size**	**Usage**
Zenmuse	JPEG	39,375	1.3 GB	Train/Val
Phantom	JPEG	8,617	301 MB	Test

Table 1: FLAME dataset splits by camera.

A key characteristic of this dataset is that the frames are derived from video footage, meaning consecutive frames are highly correlated. This results in strong redundancy within the dataset. Additionally, differences between the cameras used for training and testing introduce variation in lighting conditions and image properties. A major limitation of the dataset is the high similarity between neighbouring frames, which can introduce data leakage when frames from the same video sequence appear in both the training and validation sets. This can lead to artificially high validation performance that does not reflect true model generalisation. To address these issues, preprocessing steps were applied, including perceptual de-duplication and controlled dataset splitting, in order to reduce redundancy and improve generalisation performance.

3 Data Preprocessing

3.1 Data Splitting and de-duplication

The Dataset was pre-divided into Training and Test sets from different drone cameras. Images were organised into class folders (‘Fire‘, ‘No Fire‘), and lake images were identified by filename keyword and separated into a third logical group.

A key issue is that the images are extracted from video sequences. Consecutive frames are highly correlated, leading to significant temporal redundancy and many near-duplicate samples. To address this, we applied a de-duplication pipeline based on perceptual hashing (‘dhash‘), which captures visual similarity between images.

The images were sorted by frame number before de-duplication so that temporally adjacent frames are processed sequentially, ensuring the algorithm removes later duplicates in a video sequence. Two images were considered duplicates if their Ham-

ming distance was below a threshold of 4. Deduplication is applied independently within each class (Fire, No Fire, Lake) to avoid cross-class removal. We split the training dataset into a training and validation subset, with 15% of the training images used for validation. To do so, we used a temporal block strategy, where 1 in every 15 images was assigned to validation, distributing val samples evenly across the video timeline rather than concentrating them at the end.

After preprocessing, the final number of images in the training and validation sets is shown in Table 2. The official test set was left unchanged.

Class	Train	Validation
Fire	1640	273
No Fire	486	99
Lake	853	142

Table 2: Number of images in the training and validation sets after preprocessing and deduplication.

From this processed dataset, three classification variants were constructed.

Binary V1: Fire vs No Fire

In the first binary task, lake images were removed entirely so that the model learned only to distinguish wildfire scenes from standard non-fire environments.

Split	Fire	No Fire	Total
Train	1640	486	2126
Val	273	80	353
Test	5137	3480	8617

Table 3: Binary V1 dataset split for Fire vs No Fire, with lake images excluded.

Binary V2: Fire vs No Fire (Lake Included)

In the second binary task, lake images were retained within the No Fire class. This tests whether the presence of visually distinct water scenes biases binary classification performance.

Split	Fire	No Fire (incl. Lake)	Total
Train	1640	1339	2979
Val	273	222	495
Test	5137	3480	8617

Table 4: Binary V2 dataset split for Fire vs No Fire, with lake images retained in the No Fire class.

Ternary: Fire, No Fire, Lake

The final task treated Fire, No Fire, and Lake as three independent classes. Because the official FLAME test set contains no labelled lake images, 10% of the lake images were appended to the ternary test set and removed from train/val set.

Split	Fire	No Fire	Lake	Total
Train	1640	486	768	2979
Val	273	80	142	495
Test	5137	3480	80	8759

Table 5: Ternary dataset split for Fire, No Fire, and Lake classification.

3.2 Data Augmentation and Normalisation

All images were normalised by dividing pixel values by 255, scaling the input range from [0, 255] to [0, 1]. This helps maintain consistent gradient updates during the training phase, regardless of the dataset splits.

Data augmentation was used to enhance the model's generalisation while limiting overfitting. Images were randomly flipped in both the vertical and horizontal directions, and random brightness changes were set to 0.15. The random contrast range was set to 0.8-1.2. No augmentation was applied to the validation or test sets.

To address class imbalance in the training set,

class weights were computed using an inverse class frequency approach and passed to the model during training. This increases the penalty for misclassifying the minority class proportionally to its underrepresentation, preventing the model from defaulting to the majority class.

4 Binary Model

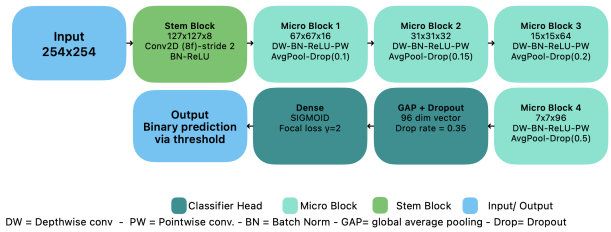

Figure 1: Lightweight convolutional neural network architecture used for binary wildfire classification. The network combines a standard stem convolution with four depthwise separable micro blocks, followed by global average pooling and a sigmoid classifier.

The model is a CNN designed for binary image classification. The main goal of the model is to be parameter-efficient for high-resolution images (254x254) while being trained on a CPU. The model should use the fewest possible parameters to train on the limited computational resources.

The full binary classification architecture is shown in Figure 1. The network uses a standard convolutional stem followed by four lightweight depthwise separable micro blocks. Spatial dimensions decrease with depth, while channel capacity increases. Global average pooling and a sigmoid classifier produce the final binary prediction.

4.1 Stem block

The input has only 3 channels (RGB), meaning depthwise separable convolution would offer negligible parameter savings at this stage. A standard Conv2D is therefore used in the stem, with depthwise separable convolutions applied in all subsequent blocks. We intentionally chose 8 filters for the stem as the input layer is very large. Having many filters would make it computationally more expensive. 8 filters extract basic low-level features (edges, textures), which are all that is needed at this early stage. Deeper layers with a smaller spatial size can afford more filters. The 3×3 kernel is the standard in modern CNNs. It captures local spatial patterns while remaining computationally cheap. The spatial resolution is halved immediately from 254×254 to 127×127, thereby reducing the computational cost of subsequent layers. After, batch normalisation was applied to normalise the output of each layer, reducing sensitivity to weight initialisation. We apply a ReLU [7] (eq.1) activation layer, which introduces nonlinearity and avoids the vanishing gradient problem associated with sigmoid and tanh activations.

$$\mathrm{ReLU}(z) = \max(0, z) \qquad \text{(eq.1)}$$

4.2 Micro Block

4.2.1 Depthwise 3×3 Convolution → Batch Normalisation → ReLU

A Depthwise convolution applies a single filter per input channel independently. This captures spatial patterns such as edges, textures, colour gradients within each feature map without mixing across channels. The Batch normalisation [8] stabilises the activation distribution, and ReLU introduces non-linearity while discarding negative activations.

4.2.2 Pointwise 1×1 Convolution → Batch Normalisation → ReLU

A pointwise Conv2D projects across channels which combines the spatially-filtered features into a new channel depth. The depthwise separable convolution which is a two step decomposition achieves 7× fewer parameters than an equivalent standard Conv2D at the same channel depth. For fire classification, where training data is limited, this parameter efficiency directly reduces overfitting risk.

4.2.3 Average Pooling, stride 2

Average pooling halves the spatial resolution at each block, progressively compressing the feature maps. Average pooling was chosen over max pooling as it produces smoother gradient flow during backpropagation and is more stable in lightweight architectures without residual connections.

4.2.4 Dropout

Dropout is applied at the end of each block at a block-specific rate. The dropout randomly zeroed activations during training to prevent co-adaptation of features. The rate increases with depth, reflecting the higher overfitting risk in later layers, where representations are more abstract and the effective dataset size is smaller due to the accumulated pooling

4.2.5 Block Progression

The four blocks are stacked sequentially, each increasing channel capacity as spatial resolution halves:

The channel depth roughly doubles at each block (16→32 → 64→96), following the established principle that as spatial resolution decreases, the channel capacity should increase to preserve the representational bandwidth.

Block	Output Shape	Dropout Rate	Rationale
1	$63 \times 63 \times 16$	0.10	Early features such as edges and colour patterns.
2	$31 \times 31 \times 32$	0.15	Mid-level textures and local structures.
3	$15 \times 15 \times 64$	0.20	More abstract feature combinations
4	$7 \times 7 \times 96$	0.25	High-level semantic representations

Table 6: Micro block output dimensions and dropout schedule used in the binary CNN architecture.

4.3 Classifier Head

The final 7×7×96 feature maps are collapsed by Global Average Pooling (GAP) into a 96-dimensional vector [9]. We use GAP because it computes the spatial average of each feature map, making the representation inherently invariant to the exact spatial location of activations, which improves generalisation to unseen images. Also, using a flattened fully connected layer would produce a 4,704-dimensional vector, and connecting this to even a modest dense layer would introduce hundreds of thousands of additional parameters, increasing the risk of overfitting. Second, GAP computes the spatial average of each feature map, making the representation inherently invariant to the exact spatial locations of activations, thereby improving generalisation to unseen images.

A dropout rate of 0.35 is applied to the 96-dimensional vector before the output layer. This is the highest dropout rate in the network and is applied here because the classification head is the point most susceptible to overfitting. A single dense unit with sigmoid (eq.2) [10] activation then yields a probability in the range (0, 1), which is thresholded to produce the final binary prediction of class 0 or class 1.

$$\text{sigmoid}(z) = \frac{1}{1 + e^{z}} \qquad \text{(eq.2)}$$

4.4 Loss Function

The Binary Focal Cross-Entropy (eq.3) was chosen as the loss function. The regular Binary Cross-Entropy loss gives equal importance to all data points; thus, data points that are easily classified and are confident do not provide any contribution to learning. To tackle this problem, Focal loss introduced a modulation term $(1-p)^{\gamma}$that reduces the loss contribution of easily classified data points and keeps the same loss contribution for data points that are difficult to classify and hence incorrectly classified. The value of γ was set to 2.0 based on the value used in the original paper [11].

$$\mathcal{L}_{\text{BFCE}} = -\alpha_t(1-p_t)^{\gamma}\log(p_t) \qquad \text{(eq.3)}$$

Class weights were applied alongside focal loss during training. Focal loss adjusts the contribution of each example based on its difficulty, while class weights adjust the contribution of each example based on its class. This ensures that the model pays additional attention to the minority class to compensate for class imbalance and remains sensitive to misclassifications in underrepresented classes.

4.5 Optimisation

The model was optimised using the Adam optimiser [12] with an initial learning rate of $\eta = 1 \times 10^{-3}$. Adam was selected over alternatives such as SGD and RMSProp as it maintains a separate adaptive learning rate for each parameter, computed from estimates of the first and second moments of the gradients. The initial learning rate of 1×10^{-3} follows the canonical default validated in the original Adam paper, balancing fast initial convergence with training stability.

Gradient Clipping using global norms was carried out with a threshold value of 1.0 to avoid exploding gradients, which could lead to disastrous parameter updating during the early stages of training when the loss surface is steep. Gradient clipping using global norms allows us to retain the direction of my gradients while restricting their value.

A ‘ReduceLROnPlateau‘ schedule was used instead of a fixed learning rate schedule. The learning rate was reduced by a factor of 0.5 whenever the validation loss failed to improve for 3 consecutive epochs. This allows smaller optimization steps once training begins to plateau. A patience of 3 helps avoid reductions caused by short-term noise, while a minimum learning rate of 1×10^{-6} prevents training from stalling completely.

Early stopping was applied with a patience of 8 epochs, monitoring validation AUC. This patience was chosen to be longer than the ‘ReduceLROnPlateau‘ patience of 3, allowing the model time to benefit from reduced learning rates before training stopped. Validation AUC was used instead of loss or accuracy because it is threshold-independent and more reliable under class imbalance, providing a better measure of class separation. When training terminated, the best weights were restored in memory and also saved using ‘ModelCheckpoint‘. Training was capped at 100 epochs, although early stopping typically ended training earlier.

4.5.1 Evaluation Metrics

The model was evaluated using four metrics: Accuracy, AUC (Area Under the ROC Curve), Precision, and Recall. While accuracy provides an intuitive measure of overall correctness, it is an unreliable indicator of performance under class imbalance, as a model can achieve high accuracy by defaulting to the majority class without meaningfully discriminating between classes. AUC was

therefore treated as the primary evaluation metric, as it measures the model's ability to rank positive examples above negative examples across all possible decision thresholds, making it both threshold-independent and robust to class imbalance. Precision and Recall were monitored alongside AUC to provide insight into the nature of any classification errors. Precision measures the proportion of positive predictions that are correct, defined as: P = TP / (TP + FP) while Recall measures the proportion of true positives that were successfully identified: R = TP / (TP + FN).

Although post-training threshold tuning was explored by searching over $t \in [0.1, 0.9]$ on the validation set, the optimal threshold generalised poorly to the test set, likely due to the validation set's small size relative to the test set. The default threshold of t = 0.5 was therefore retained for all final evaluations.

4.6 Training Behaviour

Two binary classification variants were evaluated: Binary V1, which excluded lake images from the No Fire class, and Binary V2, which retained them. Both variants were trained for a maximum of 100 epochs using identical hyperparameters and callbacks.

4.6.1 Binary V1

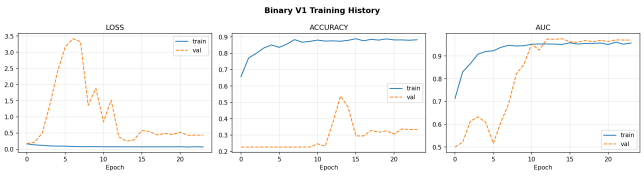

Figure 2: Training and validation performance across epochs for Binary V1.

Early stopping terminated training at epoch 24, restoring the model to its best weights at epoch 15. As Binary V1 was ultimately discarded in favour of Binary V2, a detailed analysis of its failure is provided in Section 4.7. In summary, as illustrated in Figure 2, the validation loss spiked to approximately 3.4 and remained persistently elevated throughout training, while validation accuracy plateaued at approximately 33% despite training accuracy reaching 88%. This severe train-validation gap indicates heavy overfitting driven by class imbalance when lake images were excluded. Despite this, both training and validation AUC converged to approximately 99%, confirming the model retained strong discriminative ability. This suggest that the poor accuracy reflects a miscalibrated decision threshold rather than a failure of the underlying representations.

4.6.2 Binary V2

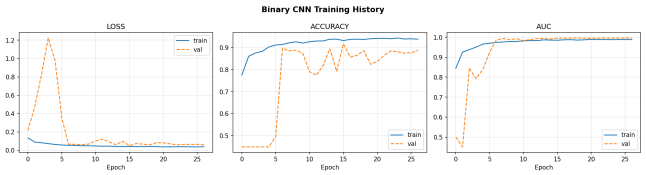

Figure 3: Training and validation performance across epochs for Binary V2.

In contrast, Binary V2 exhibited markedly more stable training dynamics, as shown in Figure 3. For V2 early stopping was triggered at epoch 27 based on validation AUC, and the model was restored to the best model at epoch 19.

At early stages, the training loss decreases smoothly and rapidly from the first epoch, whereas the validation (val) loss spikes sharply to approximately 1.2 at epoch 3 before steeply and stabilising at a low value by epoch 7. Initially, the model makes overconfident, incorrect predictions on the validation set as the weights haven't been sufficiently generalised, leading to the early validation spike. The subsequent sharp drop indicates the model correcting this behaviour once

the training loss had sufficiently constrained the weight space.

The training accuracy increases from 78% to 93%, where it plateaus mid-training. Validation accuracy remains below 20% for the first 6 epochs, then reaches 90% at epoch 7. The oscillation in validation accuracy throughout training, rather than smooth convergence, is likely due to the small validation set size (495 samples), which can cause noticeable metric fluctuations across batches. Finally, the AUC converges to 1 at the end of training for both the training and validation sets. The extremely small difference between them suggests that the model does not significantly overfit the training data in terms of discriminative ability. The gap between train and val performance is visible in the accuracy curves but absent in the AUC curves, suggesting the remaining generalisation gap is largely a threshold sensitivity issue rather than a fundamental failure of the model to separate the two classes.

4.7 Binary V1: Analysis of failure

The default treshold of probability is t=0.5, the model at best training epoch achieved a test accuracy of 50% and a validation accuracy of 47%, both equivalent to random chance for a binary task. On the test set, the model achieved a No Fire recall of 0.94 but a Fire recall of only 0.21, with a No Fire precision of just 0.45. This pattern indicates the model was predicting No Fire for the vast majority of inputs, rarely committing to a Fire prediction. On the validation set, No Fire recall reaches 1.00 while Fire recall falls to 0.32, meaning the model predicted No Fire for virtually every single image in the validation set.

This behaviour is consistent with severe class imbalance. When lake images were excluded, the No Fire class contained only 486 training images compared to 1,640 Fire images, making Fire the dominant class. Despite the application of both focal loss and class weights, the model's raw probability outputs were systematically suppressed. The predicted Fire probability rarely exceeded 0.5, causing the model to default to predicting No Fire at the standard decision boundary. This suggests the class weights and focal loss overcorrected for the imbalance, pushing the model toward excessive conservatism in its Fire predictions rather than correcting the collapse toward the majority class.

The optimal threshold identified via F1 optimisation on the validation set was $t^* = 0.19$, which is substantially below the default of $t = 0.5$. This low threshold reflects the systematic suppression of predicted Fire probabilities. We lowered the decision boundary to recover genuine Fire predictions that were being discarded at the default threshold. In other words, the model only needed 19% confidence to correctly identify Fire, indicating its probability outputs were poorly calibrated relative to the true class distribution.

When applying a threshold of t=0.19 to the validation set, we achieved a dramatic improvement, raising accuracy from 47% to 92%. The per-class F1 scores reached 0.82 and 0.95 for No Fire and Fire, respectively, and a macro-averaged F1 of 0.89. When applying the same threshold to the test set, the accuracy improved from 50% to 68%, with a Fire F1 of 0.75 and a No Fire F1 of 0.58. The gap between test and validation performance indicates that a threshold of t=0.19 partially overfits the validation set, which contains only 353 samples, leading to a failure to capture the true decision boundary. Given that even with threshold correction, Binary V1 reached only 68% accuracy, it was discarded for all subsequent evaluations.

Metric	Val	Test (t=0.5)	Test (t=0.34)	Test (t=0.34, N=3)
Accuracy	88.7%	79.0%	76.0%	86.0%
AUC	0.996	0.832	-	-
Precision (Fire)	100%	85.0%	78.6%	83.0% %
Recall (Fire)	79%	78.7%	82.7%	97.1%
Loss	0.058	0.254	-	-

Table 7: Binary V2 headline test and validation set performance (default $t = 0.50$).

4.8 Binary V2 Model Evaluation

4.8.1 Confusion Matrix Analysis

The confusion matrix for Binary V2 on the held-out test set are shown in Figure 4.

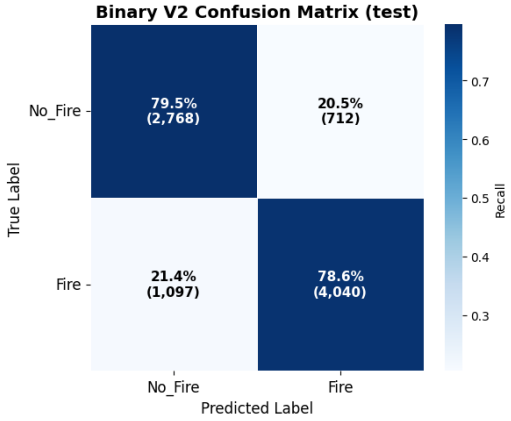

Figure 4: Confusion Matrix for Binary V2 (at default t=0.5).

Of the 3,480 No Fire images in the test set, 2,768 were correctly classified (79.5% recall) and 712 were misclassified as Fire, constituting false positives. Of the 5,137 Fire images, 4,040 were correctly identified (78.6% recall) and 1,097 were misclassified as No Fire, constituting false negatives. The model makes a similar number of mistakes in both classes, with false positives and false negatives occurring at roughly the same rate.

In wildfire detection, these two types of errors do not have the same impact. A false negative occurs when a fire image is predicted as No Fire, resulting in a real fire being missed. This could delay emergency response and lead to serious environmental and human harm. A false positive, where a No Fire image is predicted as Fire, causes a false alarm and may waste resources, but it is less dangerous. Since the model produced 1,097 false negatives in the test set, missing fires is an important weakness. For real-world use, a lower decision threshold may be better so the model detects more fires, even if this increases false alarms.

4.8.2 Threshold Analysis

Two thresholds were evaluated on the test set using boundaries derived from the validation set. The default threshold ($t = 0.50$) and the max-F1 threshold ($t = 0.34$) are summarised in Table 8. A precision-constrained threshold could not be reliably derived from the validation set due to its small size (353 images) and class imbalance (273 Fire, 80 No Fire), and was therefore excluded.

Threshold	Fire Recall	No Fire Recall	Accuracy
$t = 0.50$	78.6%	79.5%	79.0%
$t = 0.34$	82.7%	62.6%	74.6%

Table 8: Test set performance at evaluated decision thresholds.

Lowering the threshold to $t = 0.34$ increased fire recall by 4.1 percentage points, recovering an additional 206 fire images over the default. However, this came at the cost of No Fire recall falling from 79.5% to 62.6%, increasing false alarms. The consecutive-frame filter was applied at $t = 0.34$ to address this.

4.8.3 Consecutive-frame filter

To mitigate the higher number of false alarms caused by the lower threshold, the system used a sliding-window filter in the post-processing stage. The sliding window filter uses at least N consecutive frames out of W previous frames that pass the threshold test before setting off an alarm, mimicking the fact that fire persists across multiple frames.

Tuning was performed on N between 1 and 5 on the validation set, with the results illustrated in Figure 5. The system achieved a perfect recall rate with a precision of 0.892 at $N = 1$, meaning no effective filtering. With higher values of N, the algorithm filters out isolation errors, resulting in improved precision. The maximum value of F1 was 0.987, corresponding to a precision of 0.982 and a recall of 0.993 for $N = 3$. Only 5 false alarms and 2 fire misses occurred with such parameters in the validation set. Higher values of N resulted in a dramatic decrease in the recall rate, namely, 0.978 for $N = 4$ and 0.850 for $N = 5$, as the filter began rejecting fire frames. Therefore, $N = 3$ was chosen for the threshold.

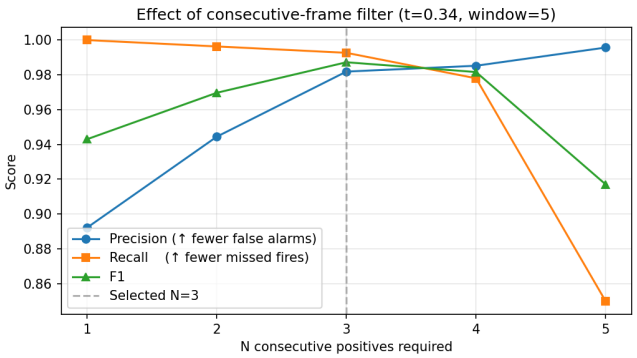

Figure 5: Precision, recall, and F1 as a function of N consecutive positives required (threshold $t = 0.34$, window $W = 5$), evaluated on the validation set. The dashed line marks the selected operating point $N = 3$.

This last system, running at $t = 0.34$ using a sequence of $N = 3$ consecutive frames from a total of $W = 5$, was tested using the test set. Confusion matrix can be seen in Figure 6. This test showed that the system could detect 97.0% of fires (4,983 of 5,137) with 154 fires missed. The No Fire recall score was 70.0%, generating 1,046 false alarms from 3,480 test No Fires. Accuracy rate was 86.0%, which is a 7 point increase from the default value.

Metric	Default ($t = 0.50$)	Final ($t = 0.34$, $N = 3$)
Accuracy	79.0%	86.0%
Fire Recall	78.6%	97.1%
Fire Precision	85.0%	83.0%
No Fire Recall	79.5%	70.3%
Missed Fires (FN)	1,097	149
False Alarms (FP)	712	1,035

Table 9: Default threshold versus final system on the held-out test set.

The final system correctly identified 97.1% of fire images (4,988 of 5,137), reducing missed fires from 1,097 to 149 (an 86% reduction). No Fire recall remained at 70.3%, with false alarms increasing moderately from 712 to 1,035. This is an acceptable trade-off, given that the cost of a missed wildfire detection is greater than the cost of a false positive.

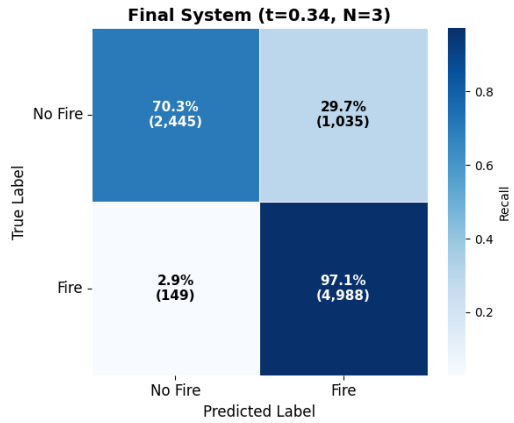

Figure 6: Confusion Matrix for Binary V2 (t=0.34, N=3 consecutive-frame filter)

4.8.4 Comparison with FLAME Baseline

The binary classifier proposed in the FLAME paper achieved 76% accuracy on the test set [6]. The accuracy of the classifier using a lower decision threshold ($t = 0.34$) and consecutive frame filters ($N = 3$) was higher by 10 percentage points (pp) at 86%. However, the difference between my model and the baseline was particularly pronounced in fire recall. We detected fire cases in 97.1% of samples, compared to 82.9% achieved by FLAME; the number of false negatives dropped

from 799 to 149, as summarised in Table 10. In the context of wildfire detection, reducing false negatives is important, as even a single missed case can lead to serious damage.

Metric	FLAME Paper	My Final System (t=0.34, N=3)	Improvement
Accuracy	76.2%	86.0%	+9.8pp
Fire Recall	82.9%	97.1%	+14.2pp
No Fire Recall	68.2%	70.3%	+2.1pp
Missed Fires (FN)	799	149	−86% fewer

Table 10: Comparison of my final system against the FLAME paper baseline on the held-out test set.

5 Ternary Model

5.1 Model Architecture

The ternary model followed the same structure as the binary CNN model; however, this time, we switched the sigmoid output to a softmax (eq.4) output [10], allowing classification across multiple categories.

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \qquad \text{(eq.4)}$$

In this case, the loss function changes to sparse categorical cross-entropy, which is standard for integer-labelled multiclass problems.

Early stopping was based on validation accuracy rather than validation AUC, since AUC is not suitable for multiclass. The rest of the callbacks stayed the same as for the Binary CNN.

The primary metric is accuracy, consistent with the multiclass setting where AUC is not straightforwardly interpretable. Top-2 accuracy was added as an additional metric, measuring whether the correct class appears among the model's top 2 predictions. Lastly, we evaluated post-training precision, recall and F1.

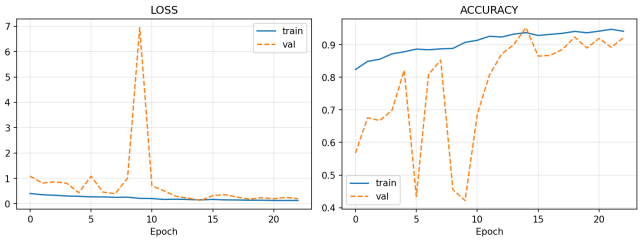

Figure 7: Training Behaviour for Ternary Model

5.2 Training Behaviour

As shown in Figure 7, for the ternary model, early stopping was triggered at epoch 18 and restored to the best weights, which were at epoch 10, when val accuracy peaked.

Training loss decreases smoothly and consistently from 0.52 to 0.24, indicating stable optimisation throughout. Val loss spiked to 1.2 at the 2nd epoch. Reflecting the same behaviour as with binary model V2, where the model makes overconfident, incorrect predictions before the weights have generalised.

The validation accuracy collapsed at epoch 3, dropping to 29%, then recovered at epoch 6 to 90%. This is due to the learning rate interacting with the class-weighted loss, temporarily pushing the model toward a suboptimal decision boundary. After epoch 6, the validation accuracy stabilised. Both training (90%) and validation (92%) accuracies converge closely and remain stable, with validation accuracy slightly exceeding training accuracy, suggesting good generalisation from the model.

The small train-val gap in the later epochs, combined with the close final values (0.90 train, 0.92 val), indicates the dropout and class weighting regularisation were effective for the ternary setting.

Metric	Val	Test (t=0.5)	Test (t=0.45)
Accuracy	95.2%	73.2%	74.3%
Top-2 Accuracy	100 %	99.1%	-
Precision (Fire)	98.1%	74.0%	73.0%
Recall (Fire)	93.4%	84.8%	85.6%
Recall (No Fire)	93.8%	55.3%	52.5%
Recall (Lake)	100%	100%	100%
Loss	0.124	0.570	-

Table 11: Ternary model headline performance across validation and test sets at default ($t = 0.50$) and tuned ($t = 0.45$) fire thresholds.

5.3 Ternary Model Evaluation

My ternary model had an initial test accuracy of 73.2% (Table 11), which is reasonable for the 3-class problem, though notably lower than the initial binary model's 79%, as expected given the added complexity of a third class. However, the top-2 accuracy of 99.1% reveals that the model's errors are almost exclusively confined to the Fire/No Fire boundary, with Fire and Lake never being mutually confused. This shows that the ternary model learned a robust separation between the two most visually distinct classes.

5.3.1 Confusion Matrix Analysis

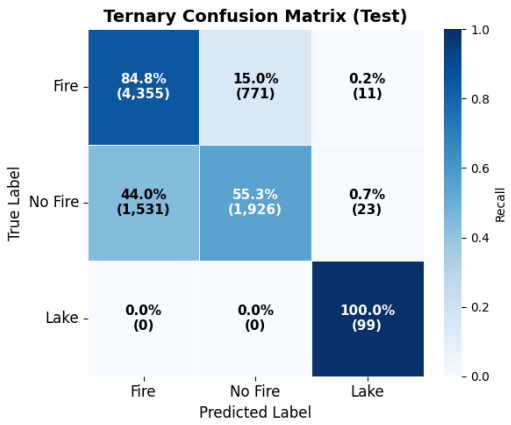

Figure 8: Confusion Matrix for Ternary Model

The confusion matrix in Figure 8 reveals the model's classification behaviour across all three classes. The lake recall is perfect at 100% across the val and test sets. The model can clearly separate water scenes from fire and vegetation, likely because of their visually distinct appearances. Fire recall is strong at 84.8% (4,355 of 5,137), with the other 15% being misclassified as No Fire, reflecting the visual ambiguity between low-intensity fire frames and non-fire vegetation. The most significant weakness is the No Fire recall at 55.3% (1,926 of 3,480), with 1,531 No Fire frames (44.0%) misclassified as Fire. This result directly highlights a limitation of the binary model: in Binary V2, lake images were retained in the No Fire class, and their visually distinct appearance likely artificially inflated No Fire recall. Once lakes are separated into their own class in the ternary setting, the remaining No Fire frames are far more visually similar to fire scenes, exposing the true difficulty of the Fire versus No Fire boundary.

5.3.2 Threshold Analysis

In the context of wildfire detection, we tried a custom Fire threshold (t) to maximise Fire recall. In this case, the threshold operates on the softmax probability of the Fire class directly: if $P(\text{Fire}) < t$, the model predicts the highest-scoring non-Fire class (No Fire or Lake) instead of defaulting to argmax.

Different threshold values were tested over the window 0.20 to 0.80 on the validation set. Where, at t=0.45, the best fire recall was achieved at 95%. When using that threshold, the overall test accuracy decreased from 73.2% to 72.5%. Lake recall remained at 100% regardless of threshold, as the fire threshold does not affect the No Fire/Lake decision boundary. Unlike the binary model, no consecutive-frame filter was applied in the ternary setting, as the improvement from threshold tuning alone was modest and the added complexity was not justified given the limited lake test set size

of 99 images. This remains a direction for future work.

5.3.3 Comparison to binary model

The ternary model achieves a test accuracy of 72.5% (at $t = 0.45$), compared to 86.0% for the final binary system, resulting in an 11.7 percentage-point reduction. This gap is expected as the ternary task is strictly harder. This increased difficulty arises because the ternary task requires the model to distinguish three classes using the same lightweight architecture. Each class also now uses a much larger training dataset.

Despite this, the ternary model provides a more honest assessment of the binary model's performance. In Binary V2, lake images were retained within the No Fire class, and their visually distinct appearance artificially inflated No Fire recall. Once lakes are separated into their own class, No Fire recall drops from 79.5% to 55.3%, indicating that a substantial portion of the binary model's No Fire performance was driven by the ease of classifying lake frames rather than by true Fire/No Fire discrimination.

Fire recall remains strong across both settings (97.1% in the binary setting, 84.8% in the ternary setting). This suggests the model can effectively identify fire. Lake classification is perfect across the board at 100% recall. This confirms that water scenes are sufficiently visually distinct to be reliably separated without any dedicated architectural changes.

6 Conclusion

In this project, we developed a lightweight micro-CNN with 11,763 parameters, built entirely from depthwise separable convolutions with no dense layers in the backbone. A key methodological contribution was the perceptual de-duplication pipeline, which reduced training data from 39,375 to approximately 2,979 frames, mitigating the temporal data leakage inherent to video-derived datasets.

My binary model V2 achieved a test accuracy of 86% and a Fire recall of 97.1% after threshold tuning ($t = 0.34$) and the application of a consecutive-frame filter ($N = 3$), reducing missed fires from 1,097 to 149 , an 86% reduction. This outperformed the FLAME paper baseline by 10 percentage points, demonstrating that simple post-processing grounded in the physical persistence of fire across frames can yield meaningful safety-critical gains.

Binary V1 further demonstrated that severe class imbalance, without lake images, led to complete model collapse, directly motivating the Binary V2 design decision to retain lake frames within the No Fire class.

The ternary classifier (t=0.45) achieved a final accuracy of 73.8%, with the new third lake category achieving 100% recall, demonstrating that the architecture can distinguish visually distinct scene types. This high recall revealed a key element of my binary model: No Fire recall dropped from 79.5% in Binary V2 to 55.3% in the ternary setting, indicating the model was good at distinguishing lake and Fire but not No fire.

No Fire generalisation remains the principal unsolved problem. The de-duplicated training set contains only 486 No Fire frames from a limited number of scenes, while the test set comprises 3,480 frames from entirely different environments captured by a different camera. This distribution shift is the ceiling on current performance, not model capacity.

To address this, future work should prioritise col-

lecting more diverse No Fire training data from varied non-fire terrain environments, revising the de-duplication strategy to retain greater scene diversity, applying the consecutive-frame filter to the ternary setting, and exploring transfer learning from pretrained backbones such as MobileNet or EfficientNet-Lite to improve No Fire generalisation without sacrificing the lightweight design goal.

Ultimately, the cost between missed fires and false alarms, where a single undetected wildfire can cause irreversible ecological and human harm, shows the importance of optimising for recall over accuracy in this domain. The systems developed here represent a step toward lightweight, deployable wildfire detection that could support real-time aerial monitoring at scale.

References

[1] Tyukavina et al. Global trends of forest loss due to fire from 2001 to 2019. *Frontiers in Remote Sensing*, 3:825190, 2022.

[2] Quentin D. Read et al. Novel fire regimes emerge from interactions of climate, land use, and invasive species. *Science*, 2020.

[3] Veronika Samborska and Hannah Ritchie. Wildfires. `https://ourworldindata.org/wildfires`, 2023. Accessed: 2026-04-15.

[4] PreventionWeb. Australia black summer bushfires 2019-2020. `https://recovery.preventionweb.net/collections/recovery-collection-australia-black-summer-bushfires-2019-2020`, 2020. Accessed: 2026-04-15.

[5] Food and Agriculture Organization of the United Nations. Global forest resources assessment 2020. `https://openknowledge.fao.org/server/api/core/bitstreams/b000416c-f865-4291-9676-77f76c4dbec1/content`, 2020. Accessed: 2026-04-15.

[6] Alireza Shamsoshoara, Fatemeh Afghah, Abolfazl Razi, Liming Zheng, Peter Fulé, and Erik Blasch. Aerial imagery pile burn detection using deep learning: The flame dataset. *Computer Networks*, 193:108001, 2021.

[7] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

[8] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[9] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. *International Conference on Learning Representations (ICLR)*, 2014.

[10] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[11] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.